

# Enforced, Fail-Closed DevSecOps Gates: Design, Threat Model, and Production Deployment Experience on AWS

Raghav Anand

Independent project · [raghavanand033@gmail.com](mailto:raghavanand033@gmail.com) · AWS production:  
[secure-cicd-dev-alb-907205721.ap-south-1.elb.amazonaws.com](https://secure-cicd-dev-alb-907205721.ap-south-1.elb.amazonaws.com)

## Abstract

*Security scanning in CI/CD pipelines is frequently advisory: a scan runs, a report is generated, and the deployment proceeds regardless of the result. This paper describes a CI/CD pipeline in which five categories of security scanning (static analysis, dependency/SCA, secret scanning, infrastructure-as-code scanning, and container scanning) are evaluated by a policy-driven,*

*fail-closed gate that blocks deployment on a threshold violation, a missing scan report, or a scanner crash. The system is deployed on real AWS infrastructure (CodePipeline, CodeBuild, ECR, ECS Fargate, ALB) provisioned by Terraform, with a supplementary API copy on Vercel. We describe the gate's design and its STRIDE-based threat model, then report, as a factual case study, eight real infrastructure and IAM failures encountered getting this system to a healthy production state — including an undocumented IAM permission found only via CloudTrail event correlation, and a distroless container health-check failure traceable to a missing binary on \$PATH. We separate this qualitative operational record from three research hypotheses about enforced-vs-advisory gating that remain unmeasured, and we state plainly which claims are and are not supported by the evidence collected so far.*

**Index Terms**—DevSecOps, CI/CD security gate, fail-closed design, shift-left security, AWS ECS Fargate, Terraform, infrastructure as code, container security.

## 1. Introduction

Security scanning bolted onto a pipeline as an advisory, non-blocking step — generate a report, maybe post it to a dashboard or a chat channel — does not change what ships; only enforcement does. This decouples detection from prevention: a scan can be perfectly accurate and still have zero effect on the deployment outcome, because nothing in the pipeline's control flow actually depends on its result. This project treats the security gate itself as a piece of software with a specification and a test suite, rather than as glue code that happens to invoke some scanners.

Concretely, the system is a small Node.js/Express API (`app/`) whose CI/CD pipeline runs five independent scanner categories on every build, evaluates their normalized JSON reports against a versioned policy file (`security/policy/security-policy.yaml`), and fails the CodeBuild phase — and therefore the entire pipeline, before the Deploy stage ever runs — on any threshold violation. There is no manual "approve to deploy" button standing between a scan result and the decision it should drive.

This is not a design exercise. The full pipeline (GitHub → CodePipeline → CodeBuild → five security gates → ECR → ECS Fargate → ALB) is deployed in a real AWS account in region `ap-south-1`, reachable at <http://secure-cicd-dev-alb-907205721.ap-south-1.elb.amazonaws.com>, with a supplementary public copy of just the API on Vercel at <https://secure-aws-cicd-demo-api.vercel.app> for quick inspection without AWS access. Building the real infrastructure surfaced eight genuine deployment failures, documented in Section 7, which this paper treats as its primary empirical contribution: not a claim about detection rates, but a factual account of what a first real AWS deployment of a security-gated pipeline actually breaks on, and how each break was root-caused.

## 2. Background and Related Concepts

**DevSecOps and shift-left security.** The general industry framing is to move security activity earlier in the development lifecycle — "shift left" — so that vulnerabilities are found at commit or build time rather than in production. In practice this is frequently implemented as tooling adoption (a SAST or SCA scanner wired into CI) without a corresponding change to the pipeline's control flow, so the scanner's output remains informational.

**Fail-open vs. fail-closed gate design.** A gate is fail-open if any ambiguous condition — a scanner that did not run, a malformed report, a network timeout fetching a scanner's rule set — defaults to allowing the deployment to proceed. A gate is fail-closed if the same ambiguous conditions default to blocking. Fail-open designs are attractive because they never block a deployment on infrastructure flakiness alone, but they are also silently bypassable: a scanner that crashes for any reason produces the same outcome as a scanner that ran and found nothing. This project's gate is fail-closed by explicit configuration (`gate.fail_closed_on_missing_report` and `gate.fail_closed_on_scanner_crash`, both true in `security-policy.yaml`).

**Policy-as-data.** Severity thresholds live in a YAML policy file, not hardcoded in the gate script, so a threshold change is a reviewable, versioned diff rather than a code change buried in pipeline logic.

## 3. System Architecture

### 3.1 CI/CD flow

A developer push to `main` triggers AWS CodePipeline via a CodeStar Connection (a tokenless, authenticated GitHub link). The Source stage hands off to a single CodeBuild project running `ci/buildspec.yml`: install dependencies, lint, run unit tests, build the Docker image, run all five scanners, evaluate the gate, and — only on a `PASS` — push the image to ECR with digest resolution and emit `imagedefinitions.json` for the Deploy stage. There is no manual approval action anywhere in this flow; the gate itself, running as ordinary CodeBuild shell commands, is the approval mechanism. A GitHub Actions workflow mirrors the identical scan-then-gate sequence for PR-time enforcement before code ever reaches `main`.

### 3.2 AWS infrastructure

Infrastructure is defined as eight modular Terraform modules (networking, IAM, ECR, ECS, CodeBuild, CodePipeline, CloudWatch, and a shared "security" module for KMS/Secrets Manager/SSM), wired together by a per-environment root module (`terraform/environments/dev`). The VPC (10.20.0.0/16) has public subnets across two availability zones carrying the ALB and a single NAT gateway, and private subnets carrying the ECS Fargate tasks with no public IP assignment. The ECS task security group accepts inbound traffic only from the ALB's security group on the container port; outbound traffic from the private subnets (image pulls, AWS API calls) routes through the NAT gateway, with no direct route to the internet gateway. A shared KMS customer-managed key encrypts ECR, Secrets Manager, CloudWatch Logs, and the pipeline's S3 artifact bucket.

The application container is built in two stages: a `node:22-alpine` build stage and a `gcr.io/distroless/nodejs22-debian12:nonroot` runtime stage, selected after comparing measured Trivy CVE counts across `node:20-alpine`, `node:22-alpine`, `node:22-slim`, and distroless candidates. CloudWatch log groups and alarms (ECS running-task count, ALB 5xx rate, ALB p95 latency) notify an SNS topic.

### 3.3 Security gate design

Evidence for the gate's decision is one normalized JSON report per scanner category, written to `security/reports/<category>/*.json` with a fixed shape: a scanner name, a status (`completed`, `tool_missing`, or `error`), a `findings` object keyed by severity, and up to 50 individual finding details. `security-gate.sh` parses `security-policy.yaml` first; if the policy itself is missing or fails to parse, the gate aborts hard with exit code 3 — the reasoning being that if the gate's own configuration is broken, nothing should be allowed to proceed on the assumption that it passed. For each enabled category, a missing report file, invalid JSON, or a status other than `completed` counts as a violation, not a skip. Secrets get a zero-tolerance rule rather than a severity threshold: any finding blocks, because a single leaked credential is unconditionally unacceptable regardless of how the scanner classified its severity. The script prints a `SECURITY GATE RESULT` table, writes a machine-readable summary, and exits 0 (pass) or 1 (fail).

## 4. Methodology: Scanners and Policy Thresholds

Five scanner categories run on every build, each with its own wrapper script that normalizes tool-specific output into the common report schema described in Section 3.3. Table I lists the scanner assigned to each category, the tool actually invoked, and the current policy threshold at which the category blocks the build, taken directly from `security/policy/security-policy.yaml`.

| Category         | Tool(s)                           | Critical/High block                 | Medium tolerated |
|------------------|-----------------------------------|-------------------------------------|------------------|
| SAST             | Semgrep                           | 0 / 0                               | 10               |
| Dependency / SCA | npm audit, OWASP Dependency-Check | 0 / 0                               | 5                |
| Secrets          | Gitleaks                          | any finding blocks (zero tolerance) | n/a              |
| IaC              | Checkov (terraform/)              | 0 / 0                               | 15               |
| Container        | Trivy                             | 0 / 0                               | 20               |

TABLE I. Scanner categories and blocking thresholds (`security-policy.yaml`, version 1). "Low" findings are uncapped (policy value -1) in every category and never block.

A SBOM (Software Bill of Materials) is also generated per build by Syft, in both CycloneDX and SPDX formats (`ci/scripts/generate-sbom.sh`); this is currently best-effort and informational only, not a gate input — turning "a new component appeared with no SBOM-diff review" into a blocking rule is listed as future work.

The gate's decision logic is exercised by its own fixture-based test suite (`tests/security/run-tests.sh`), covering seven scenarios: pass when all categories are within policy; fail on a severity threshold breach; fail on any secret finding; and fail closed on a missing report, a malformed (non-JSON) report, a scanner-crashed status, and a tool-not-installed status. All seven pass. This establishes that the gate's mechanism behaves as designed — it does not, by itself, establish anything about detection or enforcement rates over real pipeline executions, a distinction Section 8 returns to.

## 5. Threat Model Summary

The project maintains a STRIDE-based threat model covering 14 threats across the application, the CI/CD pipeline, and the AWS infrastructure (`docs/threat-model.md`), each with an attack vector, affected component, existing control, and a qualitative residual-risk rating (no quantitative likelihood or detection-rate figure is asserted anywhere in that document). The threats span both classic application-layer concerns (vulnerable dependencies, leaked credentials, malicious commits) and pipeline/infrastructure-specific concerns that are easy to omit from a narrower threat model: a compromised CodeBuild environment pivoting on the CodeBuild IAM role's permissions; artifact tampering between the Build and Deploy stages, mitigated by ECR's immutable tags and digest-pinned deployment rather than mutable-tag deployment; and a privileged actor bypassing the gate entirely by editing `buildspec.yaml` or the policy file, mitigated only by code review and branch protection, not by any technical control internal to the repository.

The single largest documented gap relative to a production-grade deployment is the absence of AWS CloudTrail and AWS Config: without account-level audit logging, an insider with legitimate AWS console or CLI access could modify infrastructure outside the pipeline with no automatic record. The threat model names this explicitly rather than omitting it. A dedicated supply-chain subsection covers SBOM generation, digest pinning, lockfile pinning, and pinned (non-`latest`) base images; it states plainly that image signing and SLSA provenance attestation are not implemented and are the most significant remaining supply-chain gap.

## 6. Production Deployment and Failure Analysis

Deploying this project's Terraform to a real AWS account (`ap-south-1`) surfaced eight genuine, deploy-blocking failures before the pipeline reached a healthy, ALB-served ECS Fargate task. This is presented as a complete list, not a curated highlight reel; every root cause listed was confirmed against real AWS API output before a fix was written. Table II summarizes all eight; Sections 6.1 and 6.2 give the two most novel in full detail.

| # | Symptom                                                           | Root cause                                                                      | Found via                    |
|---|-------------------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------|
| 1 | terraform apply: InvalidParameterValue on subnet AZs              | Hardcoded us-east-1a/1b AZs don't exist in ap-south-1                           | apply error output           |
| 2 | 4 CloudWatch log groups: AccessDeniedException                    | KMS key policy granted account root, not the logs service principal             | apply error output           |
| 3 | CodeBuild: Not authorized to DescribeSecurityGroups               | VPC-mode CodeBuild needs explicit EC2 ENI-management IAM, not implied elsewhere | CodeBuild build log          |
| 4 | CodeBuild: YAML_FILE_ERROR on a buildspec valid in GH Actions     | CodeBuild's YAML parser is stricter; unquoted ':' read as a nested key          | CodeBuild parse failure      |
| 5 | CodeBuild: yq: command not found                                  | gate script needs yq; CodeBuild managed image lacks it (GH Actions ships it)    | CodeBuild build log          |
| 6 | CodePipeline Deploy: generic PermissionError x3                   | ecs:TagResource required but absent from AWS's own reference IAM policy         | CloudTrail lookup-events     |
| 7 | ECS circuit breaker: tasks failed to start                        | Re-registered task def used bootstrap default tag ":initial", never pushed      | ecs describe-services events |
| 8 | ECS task UNHEALTHY and killed at ~7 min despite ALB seeing 200 OK | Exec-form health check ran bare "node"; distroless \$PATH excludes /nodejs/bin  | app logs + docker inspect    |

TABLE II. All eight production incidents encountered deploying to a real AWS account (`docs/production-incidents.md`).

### 6.1 Incident 8: the distroless \$PATH health-check bug

ECS reported the deployment's rollout state as FAILED with reason "ECS deployment circuit breaker: tasks failed to start," and the service event log showed the task failing container health checks and being stopped. The misleading part: the ALB's own external health check (`ELB-HealthChecker/2.0`) was hitting `/health` on the task's private IP every 15 seconds and getting 200 OK for the task's entire seven-plus-minute lifetime — so the ALB target group and a manual curl both looked completely healthy.

The actual investigation pulled the task's CloudWatch log stream directly and grepped every `/health` request by its user agent:

100% of the requests logged came from ELB-HealthChecker/2.0; not one ever came from the container's own internal health-check process. That absence was the actual signal — ECS's container-level `healthCheck` was never reaching the application process at all.

The root cause was confirmed directly with `docker inspect` against

`gcr.io/distroless/nodejs22-debian12:nonroot`:

```
$ docker inspect
gcr.io/distroless/nodejs22-debian12:nonroot \
  --format '{{json .Config.Entrypoint}}'
["/nodejs/bin/node"]
$ docker inspect
gcr.io/distroless/nodejs22-debian12:nonroot \
  --format '{{json .Config.Env}}'
["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin", "SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt"]
```

The image's `ENTRYPOINT` is an absolute path, so the container's main process starts fine without needing a `$PATH` lookup. But an `exec-form HEALTHCHECK/ECS healthCheck.command` is a separate `exec()` resolved via `$PATH` exactly like a shell would, and this image's `$PATH` does not include `/nodejs/bin`. Every health-check invocation failed with an `executable-not-found` condition before it ever opened a TCP connection — consistent with the log evidence of zero internal health-check requests. The fix uses the same absolute path in both the Dockerfile `HEALTHCHECK` and the ECS task definition's `healthCheck.command`:

```
HEALTHCHECK --interval=30s --timeout=5s \
  --start-period=10s --retries=3 \
  CMD ["/nodejs/bin/node", "-e", "..."]
```

This generalizes beyond this one image: any `distroless` or otherwise `shell-less`, `minimal-$PATH` base image will hit the identical failure mode for any `exec-form` health check that assumes the `entrypoint` binary is reachable on `$PATH`, since this is a property of how Docker/ECS execute `CMD-form` health checks, not something specific to Node.js or to this image.

## 6.2 Incident 6: an undocumented `ecs:TagResource` permission

AWS publishes a documented reference IAM policy for CodePipeline's ECS deploy action. Two consecutive deployment attempts matched that reference exactly — once with a narrowly scoped custom policy, once by copying AWS's own reference policy verbatim — and both still failed with the same generic "not sufficient permissions to access ECS" message, which CodePipeline does not expand with the specific denied action.

`aws cloudtrail lookup-events`, filtered for `AccessDenied` events around the failed execution's timestamp, surfaced the actual denial:

```
User: arn:aws:sts::<account>:assumed-role/
  <name>-codepipeline-role/<session>
is not authorized to perform: ecs:TagResource
on resource: arn:aws:ecs:<region>:<account>:
  task-definition/<name>-app:*
because no identity-based policy allows the
ecs:TagResource action
```

CodePipeline's ECS deploy action propagates the task definition family's tags onto every new revision it registers — an implementation detail not present in AWS's own published

reference policy for this feature. The fix was a single scoped IAM statement granting `ecs:TagResource` on the task definition family ARN. The operational lesson: for an opaque IAM `AccessDenied` error with no specific action named, CloudTrail's raw event log proved more reliable than the cloud provider's own "complete" reference policy.

None of the eight incidents were security vulnerabilities in the conventional sense — no injected malicious code, no exploited misconfiguration. They were operational-correctness bugs that a first real deployment to a fresh AWS account reliably surfaces: region-specific assumptions, implicit IAM permission requirements a cloud provider's own documentation omits, and a health-check assumption that silently breaks under a specific, increasingly common, security-motivated container-image choice. Finding and fixing all eight before the service was considered "working" is itself supporting evidence for this project's broader position: verification has to be adversarial against real system behavior, not just a `terraform apply` exit code of 0.

## 7. Research Framing: Hypotheses vs. What Has Been Measured

This project doubles as a research artifact on whether automated, fail-closed enforcement changes deployment outcomes relative to advisory scanning. Three research questions are posed: whether an automated, fail-closed gate measurably changes the rate at which vulnerable code reaches a deployable state versus an advisory gate (RQ1); what the time and false-positive cost of a five-category scan suite is, relative to catching the same issues later in production incident response (RQ2); and whether a fail-closed design measurably reduces deployments that proceed despite incomplete security evidence, without an unacceptable increase in false-positive-driven pipeline failures (RQ3).

These map to three hypotheses. **H1**: an automated fail-closed gate blocks a materially higher proportion of commits containing an injected `CRITICAL/HIGH` vulnerability than an advisory-only gate running the same scans. **H2**: the added wall-clock time of running all five scanner categories is small relative to total pipeline execution time (dominated by dependency install and Docker build), so the gate does not become the primary bottleneck. **H3**: a fail-closed design blocks 100% of "scanner didn't actually run" cases that a fail-open design would silently pass, at some cost in false-positive pipeline failures from transient scanner installation or network issues.

The fixture-based test suite in Section 4 establishes H3's *logical* correctness — the gate code, as written, does fail closed on every fixture scenario representing a scanner that did not actually run. That is a distinct, narrower claim from H3's *operational* form, which is about the frequency and cost of that behavior across many real pipeline runs over time, and remains unmeasured. H1 and H2 are entirely unmeasured: no controlled comparison between a Gated and an Advisory pipeline configuration, and no measured scan-overhead distribution, has been run.

**What has actually been measured.** The infrastructure this methodology targets has been deployed to a real AWS account, and the pipeline has run to completion — Source, Build, all five gate categories, Deploy — multiple times against real commits. This produced one piece of genuine empirical data: the eight-incident case study in Section 6. That is a qualitative

operational case study, not a run of the DBR/VDR/FPR/MTTD/MTTR methodology defined for H1–H3 (Table III), and it says nothing about detection or enforcement rates for injected vulnerabilities. What it does support is a more modest claim: that this system's gate and infrastructure were exercised against real, unscripted AWS behavior — region-specific defaults, IAM permission requirements undocumented by the provider, a distroless image's \$PATH behavior — rather than only a terraform plan and a local test suite. The controlled-corpus trial (deliberately injected vulnerabilities per category, Gated vs. Advisory configurations, repeated runs) described in the methodology has not been run.

| Metric                   | Definition (abridged)                                     | Status  |
|--------------------------|-----------------------------------------------------------|---------|
| DBR                      | % injected-vuln commits blocked from Deploy               | Not run |
| VDR                      | % injected vulns flagged by ≥ 1 scanner                   | Not run |
| FPR                      | % clean commits blocked anyway                            | Not run |
| MTTD                     | Push to first flagging report                             | Not run |
| MTTR                     | Gate FAIL visible to corrected commit passing             | Not run |
| Scan Overhead            | Added wall-clock time of all 5 scans                      | Not run |
| Fail-Closed Trigger Rate | % failures from missing/crashed report vs. real violation | Not run |

TABLE III. Metrics defined by the methodology (docs/research-notes.md) and their measurement status as of this writing. No figure for any of these metrics is asserted anywhere in this project.

No specific DBR, VDR, FPR, MTTD, MTTR, or timing figure is stated anywhere in this paper or the underlying repository as a measured result. H1–H3 remain reasoned expectations based on the gate's design, not conclusions; a rigorous execution of the methodology would report whichever hypotheses the data actually supports, including the possibility that none hold as stated.

## 8. Limitations

- No DAST, fuzzing, or formal penetration testing.
- No CloudTrail or AWS Config deployed — the largest documented gap versus a production-grade deployment.
- No automatic Secrets Manager rotation (manual rotation steps only).
- No image signing or SLSA provenance attestation.
- Single NAT gateway, not multi-AZ-redundant (cost-optimized for a demo/student account).
- ALB serves plain HTTP; no ACM certificate or custom domain is provisioned.
- SBOM generation (Syft) is best-effort and informational, not a gate input.
- In-memory user store with no persistence across process/task restarts, unreliable across Vercel serverless cold starts specifically.
- No frontend/UI and no AI/ML component — the engineering focus is deliberately the pipeline, not the application.

## 9. Future Work

- Wire SBOM diffing into the gate: block on a new, unreviewed component appearing.

- Image signing (cosign/Sigstore) and SLSA provenance attestations, verified before ECS deploy.
- Deploy CloudTrail and AWS Config for a full account-level audit trail.
- Automatic Secrets Manager rotation via a Lambda rotation function.
- Multi-AZ NAT gateways and an HTTPS listener with ACM for a production-grade (not just demo-grade) deployment.
- Rootless/Kaniko-style container builds to remove CodeBuild's privileged-mode requirement.
- Actually execute the controlled-corpus methodology of Section 7 against real pipeline runs and report measured DBR, VDR, FPR, MTTD, and MTTR figures.

## 10. Conclusion

This paper described a CI/CD pipeline in which security scanning is an enforced, fail-closed gate rather than an advisory report, deployed on real AWS infrastructure and exercised through eight genuine production incidents — region-specific configuration errors, undocumented IAM permission requirements, and a distroless container health-check failure traceable to a missing entry on \$PATH — each root-caused with real AWS API evidence rather than assumption. The gate's fail-closed decision logic is verified by a fixture-based test suite as logically correct. The project explicitly does not claim that this logical correctness, or the incident case study, measures the enforcement-versus-advisory effect that motivates its three research hypotheses; that measurement requires the controlled-corpus trial described in Section 7 and left as future work. The distinction between what has been built and verified, what has been observed in a real deployment, and what remains an untested hypothesis is treated throughout as the load-bearing contribution of this work.

## References

- [1] OWASP Foundation, “OWASP Top 10,” [owasp.org/www-project-top-ten/](https://owasp.org/www-project-top-ten/). General reference for common web application vulnerability classes relevant to the SAST/SCA scan categories.
- [2] National Institute of Standards and Technology, “Secure Software Development Framework (SSDF), NIST SP 800-218,” [csrc.nist.gov](https://csrc.nist.gov). General reference for secure software development practices, including build-pipeline integrity.
- [3] Amazon Web Services, “AWS Well-Architected Framework — Security Pillar,” [aws.amazon.com/architecture/well-architected/](https://aws.amazon.com/architecture/well-architected/). General reference for the AWS infrastructure design principles (least privilege, defense in depth, encryption at rest/in transit) applied in Section 3.2.
- [4] Amazon Web Services, AWS CodePipeline and AWS CodeBuild documentation, [docs.aws.amazon.com](https://docs.aws.amazon.com). Referenced for the managed CI/CD services this pipeline is built on and for the ECS-deploy reference IAM policy discussed in Section 6.2.
- [5] Google Cloud, “distroless container images,” [github.com/GoogleContainerTools/distroless](https://github.com/GoogleContainerTools/distroless). Referenced for the runtime base image discussed in Section 6.1.